# CSC101: APPLICATIONS OF ICT
# FINAL LAB PROJECT
# CAFÉ BILLING SYSTEM


Submitted by

| Name | Registration ID |
|---|---|
| **Sohayel Mahmud** | SP26-BCS-114 |
| **Tauhidur Rahman Shishir** | SP26-BCS-119 |
| **Md Ehtiyaz Tawsif Abid** | SP26-BCT-062 |

Submitted to

Musab Saeed
Visiting Faculty
Department of Computer Science
COMSATS University Islamabad



JUNE 12, 2026

COMSATS UNIVERSITY ISLAMABAD
Park Road, Tarlai Kalan, Islamabad 45550, Pakistan

# Table of Contents

.....0
Project Description.....2
Task Distribution .....2
Features of System.....2
Modules and Entities .....3
1. Main Execution Module .....3
2. Menu Display & Order Collection Module (process_order) .....3
3. Item Data Lookup Module (get_item_data) .....4
4. Bill Calculation & Discount Module .....4
5. Invoice Generation Module (print_invoice).....4
6. Input Validation Module (get_valid_int).....4
Source Code .....5
Screenshots of All Possible Outputs.....8
Main menu: .....8
Select Items: .....8
Bill Generate:.....9
Exiting from the System:.....9
Conclusion .....10

## Project Description

The Cafe Management & Billing System is a console-based Python application designed to streamline and automate the order-taking and invoice generation process within a cafe environment. The system provides a fast, structured, and user-friendly platform where cafe staff can process customer orders one by one in a continuous operational loop, making it suitable for real-time counter use. Unlike systems requiring login credentials or administrative dashboards, this application is built around a single, focused workflow — presenting the menu, collecting item selections with quantities, computing the bill, applying applicable discounts, and displaying a fully formatted invoice on the terminal at the end of each transaction.

The menu consists of seven items spanning both beverages and food, with prices fixed within the system. Each order session begins with a clear display of the menu card, after which the operator enters Item IDs and quantities for the customer. The system intelligently handles duplicate item entries by merging quantities rather than creating redundant entries, ensuring a clean and accurate cart at all times. Once the customer finalizes their selection, the bill is computed automatically, a discount is applied if the subtotal qualifies, and a uniquely identified invoice is printed to the screen with a complete itemized breakdown.

All numeric inputs across the entire application are protected by a centralized validation function that enforces both data type correctness and acceptable value ranges, ensuring the program remains stable and does not crash under any user input scenario. After each customer's invoice is printed, the operator is prompted to either process the next customer or exit the system gracefully, allowing continuous uninterrupted service throughout the cafe's operating hours.

## Task Distribution

| Team Member | Registration ID | Responsibilities |
|---|---|---|
| **Sohayel Mahmud** | SP26-BCS-114 | Item Data Lookup Function (get_item_data), Input Validation Helper Function (get_valid_int), Main Program Loop & Exit Control (main) |
| **Tauhidur Rahman Shishir** | SP26-BCS-119 | Menu Display, Order Collection Loop, Cart Management & Duplicate Item Merging Logic, Real-Time Subtotal Tracking (process_order) |
| **MD. Ehtiyaz Tawsif Abid** | SP26-BCT-062 | Invoice Generation & Formatting, Unique Order ID Generation, Bill Calculation, Discount Application Logic (print_invoice) |

## Features of System

- **Continuous Multi-Customer Processing:** The system operates in a persistent loop, allowing the operator to process an unlimited number of customers back-to-back in a single session without restarting the program.
- **Structured Menu Display:** A clearly formatted menu card is presented at the start of every order session, listing all seven items with their respective IDs and prices in a consistent, easy-to-read layout.

- **Item ID Based Ordering:** Customers place orders by selecting items through numeric IDs, reducing the chance of input errors and enabling fast order entry suitable for a counter environment.
- **Smart Cart Management:** If the same item is ordered more than once within a single session, the system automatically merges the quantities into a single cart entry rather than duplicating the item, keeping the invoice clean and accurate.
- **Automated Bill Calculation:** The system computes the subtotal in real-time after each item addition and displays the running total to the operator, providing instant visibility into the order value as it grows.
- **Automatic Discount Application:** A 10% flat discount is automatically applied to any order whose subtotal exceeds PKR 1,000, with the discount amount and final net payable clearly separated and displayed on the invoice.
- **Formatted Invoice Generation:** Upon order completion, a neatly bordered, column-aligned invoice is printed to the terminal containing the unique Order ID, itemized list with quantities and totals, subtotal, discount, and net payable amount.
- **Random Unique Order ID:** Every confirmed invoice receives a randomly generated Order ID prefixed with "CF" (e.g., CF47), providing a simple reference identifier for each transaction.
- **Centralized Input Validation:** A single reusable get_valid_int() function handles all numeric inputs throughout the system, enforcing both data type correctness and range boundaries with clear, descriptive error messages on every violation.
- **Graceful Exit Control:** After each customer's invoice is processed, the operator is given a binary choice to either continue with the next customer or exit the system, which terminates with a polite goodbye message.
- **Lightweight & Dependency-Free:** The application uses only Python's built-in random module with no external libraries or file system dependencies, making it instantly runnable on any standard Python installation without any setup.

## Modules and Entities

### 1. Main Execution Module

Controls the top-level program loop that drives continuous customer processing. After each invoice is printed, it prompts the operator to process the next customer or exit. Calls process_order() for each new transaction and terminates cleanly on exit selection.

### 2. Menu Display & Order Collection Module (process_order)

Handles the complete customer order workflow from start to finish. Displays the formatted menu card at the beginning of each session, collects Item ID and quantity inputs in a loop, merges duplicate item entries into the existing cart, maintains a running subtotal, and triggers bill finalization and invoice printing once the operator signals the end of the order.

### 3. Item Data Lookup Module (get_item_data)

A dedicated function that maps a numeric Item ID to its corresponding item name and unit price. Acts as a centralized data reference for all menu items, keeping item information consistent across the order collection and invoice generation stages without repeating data in multiple places.

### 4. Bill Calculation & Discount Module

Computes the final bill from the accumulated subtotal at the end of each order. Checks whether the subtotal exceeds PKR 1,000 and, if so, calculates a 10% discount automatically. Produces the three billing values — subtotal, discount, and net payable — that are passed directly to the invoice printing function.

### 5. Invoice Generation Module (print_invoice)

Generates and displays a fully formatted terminal invoice for every completed order. Produces a random unique Order ID, prints an itemized table of all ordered items with quantities and line totals, and displays the billing summary including subtotal, discount, and final net payable amount. Concludes with a thank-you message to the customer.

### 6. Input Validation Module (get_valid_int)

A reusable utility function called at every point in the system where numeric input is required. Validates that the entered value is a positive integer and falls within a specified minimum and maximum range. Displays appropriate error messages for both invalid data types and out-of-range values, looping until a valid input is received.

## Source Code

```
import random

def get_valid_int(prompt, min_val, max_val):
    while True:
        user_input = input(prompt).strip()
        if user_input.isdigit():
            val = int(user_input)
            if min_val <= val <= max_val:
                return val
            else:
                print(f"Invalid range! Please enter between {min_val} and {max_val}.")
        else:
            print("Invalid input! Please enter a valid positive integer number.")

def print_invoice(ordered_items, ordered_qtys, subtotal, discount, final_bill):
    order_id = "CF" + str(random.randint(1, 100))

    print("\n====================================")
    print(f"       INVOICE - ORDER #{order_id}   ")
    print("====================================")
    print(f"{'Item Name':<20} | {'Qty':<5} | {'Total':<10}")
    print("------------------------------------")

    for i in range(len(ordered_items)):
        name = ordered_items[i]
        qty = ordered_qtys[i]
        if name == "Espresso": price = 250
        elif name == "Americano": price = 280
        elif name == "Cappuccino": price = 320
        elif name == "Cafe Latte": price = 350
        elif name == "Chocolate Brownie": price = 180
        elif name == "Club Sandwich": price = 420
        elif name == "Burger": price = 280
        else: price = 0

        print(f"{name:<20} | {qty:<5} | {price * qty:<10}")

    print("------------------------------------")
    print(f"Subtotal:     {subtotal} PKR")
    print(f"Discount:     {discount} PKR")
    print(f"Net Payable:  {final_bill} PKR")
    print("====================================")
    print("     Thank you for your visit!     ")

def get_item_data(item_id):
    if item_id == 1:
        return "Espresso", 250
    elif item_id == 2:
```

```
        return "Americano", 280
    elif item_id == 3:
        return "Cappuccino", 320
    elif item_id == 4:
        return "Cafe Latte", 350
    elif item_id == 5:
        return "Chocolate Brownie", 180
    elif item_id == 6:
        return "Club Sandwich", 420
    elif item_id == 7:
        return "Burger", 280
    return "", 0

def process_order():
    print("\n====================================")
    print("          CAFE MENU CARD            ")
    print("====================================")
    print("ID 1 | Espresso             | 250 PKR")
    print("ID 2 | Americano            | 280 PKR")
    print("ID 3 | Cappuccino           | 320 PKR")
    print("ID 4 | Cafe Latte           | 350 PKR")
    print("ID 5 | Chocolate Brownie    | 180 PKR")
    print("ID 6 | Club Sandwich        | 420 PKR")
    print("ID 7 | Burger               | 280 PKR")
    print("====================================")

    ordered_items = []
    ordered_qtys = []
    subtotal = 0

    while True:
        item_id = get_valid_int("Enter Item ID to order (0 to finish): ", 0, 7)

        if item_id == 0:
            break

        qty = get_valid_int("Enter Quantity (1-10): ", 1, 10)

        item_name, price = get_item_data(item_id)

        if item_name in ordered_items:
            idx = ordered_items.index(item_name)
            ordered_qtys[idx] += qty
        else:
            ordered_items.append(item_name)
            ordered_qtys.append(qty)

        subtotal += price * qty
        print(f"Added {qty}x {item_name} to cart. Current Subtotal: {subtotal} PKR\n")
```

```
    if len(ordered_items) == 0:
        print("No items selected.")
        return

    discount = 0
    if subtotal > 1000:
        discount = int(subtotal * 0.10)

    final_bill = subtotal - discount

    print_invoice(ordered_items, ordered_qtys, subtotal, discount, final_bill)

def main():
    while True:
        process_order()
        print("\n-----------------------------------")
        choice = get_valid_int("Process next customer? (1 for Yes, 0 to Exit): ", 0, 1)
        if choice == 0:
            print("Thank you for using Cafe System. Goodbye!")
            break

if __name__ == "__main__":
    main()
```

# Screenshots of All Possible Outputs

## Main menu:

```
PS D:\BCS-Sem-1\CODES\ICT\FinalProject> & "C:/Users

==================================
          CAFE MENU CARD
==================================
ID 1 | Espresso             | 250 PKR
ID 2 | Americano            | 280 PKR
ID 3 | Cappuccino           | 320 PKR
ID 4 | Cafe Latte           | 350 PKR
ID 5 | Chocolate Brownie    | 180 PKR
ID 6 | Club Sandwich        | 420 PKR
ID 7 | Burger               | 280 PKR
==================================
Enter Item ID to order (0 to finish):
```

Showing the Full Menu

## Select Items:

```
==================================
Enter Item ID to order (0 to finish): 1
Enter Quantity (1-10): 2
Added 2x Espresso to cart. Current Subtotal: 500 PKR

Enter Item ID to order (0 to finish): 2
Enter Quantity (1-10): 3
Added 3x Americano to cart. Current Subtotal: 1340 PKR

Enter Item ID to order (0 to finish): 3
Enter Quantity (1-10): 4
Added 4x Cappuccino to cart. Current Subtotal: 2620 PKR

Enter Item ID to order (0 to finish): 4
Enter Quantity (1-10): 5
Added 5x Cafe Latte to cart. Current Subtotal: 4370 PKR

Enter Item ID to order (0 to finish): 5
Enter Quantity (1-10): 6
Added 6x Chocolate Brownie to cart. Current Subtotal: 5450 PKR

Enter Item ID to order (0 to finish): 6
Enter Quantity (1-10): 7
Added 7x Club Sandwich to cart. Current Subtotal: 8390 PKR

Enter Item ID to order (0 to finish): 7
Enter Quantity (1-10): 1
Added 1x Burger to cart. Current Subtotal: 8670 PKR

Enter Item ID to order (0 to finish):
Invalid input! Please enter a valid positive integer number.
Enter Item ID to order (0 to finish): 0
```

Customer selecting the items & their quantity

## Bill Generate:

```
==================================
       INVOICE - ORDER #CF46
==================================
Item Name            | Qty   | Total
----------------------------------
Espresso             | 2     | 500
Americano            | 3     | 840
Cappuccino           | 4     | 1280
Cafe Latte           | 5     | 1750
Chocolate Brownie    | 6     | 1080
Club Sandwich        | 7     | 2940
Burger               | 1     | 280
----------------------------------
Subtotal:     8670 PKR
Discount:     867 PKR
Net Payable:  7803 PKR
==================================
     Thank you for your visit!
```

Showing the Calculated bill for each item, Subtotal, Calculating Discount and then the final Amount to pay.

## Exiting from the System:

```
----------------------------------
Process next customer? (1 for Yes, 0 to Exit): 0
Thank you for using Cafe System. Goodbye!
PS D:\BCS-Sem-1\CODES\ICT\FinalProject>
```

Asking for if generate another Bill or exiting.

## Conclusion

The Cafe Management & Billing System successfully fulfills all the objectives set out for this project by delivering a fully functional, menu-driven console application built in Python. The system effectively implements a streamlined single-role workflow centered entirely around speed, simplicity, and accuracy — enabling cafe counter staff to process multiple customers back-to-back in a single uninterrupted session without any administrative overhead or login requirements. All core operations — menu display, item selection, cart management, bill calculation, discount application, and invoice generation — have been implemented with robust input validation and clean modular design, ensuring the application remains stable and crash-free under all user input scenarios.

The use of Python lists to maintain the ordered items and quantities during each session demonstrates a solid understanding of in-memory data management and runtime state tracking. The smart cart merging logic, which consolidates duplicate item entries by updating quantities rather than creating redundant records, reflects careful attention to real-world usability in a counter environment. The centralized get_valid_int() validation function, applied consistently at every input point throughout the system, exemplifies the value of reusable, modular code design in building reliable applications.

The automated discount calculation, real-time subtotal feedback during ordering, and the cleanly formatted terminal invoice with a randomly generated unique Order ID together add meaningful practical utility to the system, making it a realistic representation of a lightweight point-of-sale tool. The lightweight, dependency-free architecture further ensures that the application can be deployed and run instantly on any standard Python installation without configuration or setup.

Overall, this project provided valuable hands-on experience in applying core programming fundamentals — including conditional logic, list manipulation, modular function design, loop control, and input validation — in the context of a practical, real-world application that simulates an operational cafe billing environment.